PLC Firmware Platform Manual

Guía técnica oficial del proyecto PLC con ESP32-S3, HMI compacta y arquitectura modular industrial

Versión del documento: 0.9 (base de ingeniería)
Firmware objetivo: plc_firmware · Target: ESP32-S3-WROOM-1
Estado documental: diseño e integración temprana

Propósito: Este manual documenta la arquitectura, organización, compilación, operación y evolución del firmware PLC. Está pensado como documento principal de desarrollo, integración, mantenimiento y escalabilidad.

1. Título del manual

Nombre propuesto del manual: PLC Firmware Platform Manual — ESP32-S3 Industrial Control Stack.

Descripción breve del proyecto: plataforma de firmware para un PLC compacto basado en ESP32-S3-WROOM-1, con HMI táctil de 320 px, drivers industriales, servicios de configuración, comunicaciones y un núcleo PLC extensible.

El propósito del documento es servir como guía técnica oficial del sistema, con enfoque equivalente a una guía tipo Get Started, pero adaptada a una plataforma PLC/HMI propia.

2. Introduction

El proyecto implementa una plataforma de control programable compacta orientada a automatización industrial ligera y mediana. La base del sistema es un ESP32-S3-WROOM-1, seleccionado por su equilibrio entre capacidad de procesamiento, conectividad, ecosistema de desarrollo y soporte para interfaces ricas de usuario.

El objetivo principal es consolidar en un solo firmware cuatro dominios que normalmente aparecen separados: control de entradas y salidas, interfaz HMI local, conectividad industrial y servicios de configuración remota. De esta forma, el PLC no se limita a ejecutar lógica de control, sino que expone diagnóstico, configuración, mantenimiento y capacidad de ampliación.

El problema que este proyecto resuelve es la necesidad de contar con una base de firmware modular, mantenible y escalable antes de cerrar el hardware definitivo. La arquitectura actual permite avanzar el software, compilar, integrar módulos y documentar el comportamiento incluso cuando parte del PCB, algunos buses o varias interfaces siguen pendientes de validación.

- Target principal: ESP32-S3-WROOM-1.
- Arquitectura por componentes en ESP-IDF.
- Drivers para buses, E/S digitales, analógicas, energía, SD y USB device.
- Servicios de red y configuración: Wi-Fi, web config y BLE config.
- Comunicaciones industriales: Modbus, MQTT y bridges para OpenPLC / 4Diac.
- HMI local compacta con menú horizontal y estética industrial oscura.

3. What You Need

3.1 Hardware

Elemento	Rol	Estado
ESP32-S3-WROOM-1	MCU principal del sistema	Obligatorio
Pantalla HMI compacta	Interfaz local de operación	Obligatorio para HMI
Tarjeta mezzanine	Interconexión de pantalla, USB y card	Por validar en hardware final
Interfaz USB	Programación, diagnóstico o modo device	Implementación base definida
SD card	Logs, backup y archivos locales	Opcional/recomendado

Fuente de alimentación	Alimentación estable del módulo y periféricos	Obligatorio
Entradas digitales	Señales NPN y PNP	Obligatorio en PLC final
Salidas digitales	Salidas NPN, PNP y relés	Obligatorio en PLC final
Módulos analógicos	ADC externo, AO 4–20 mA y AO 0–10 V	Según variante de hardware
Cable USB / serial	Flasheo, monitor y depuración	Obligatorio para desarrollo

3.2 Software

Componente	Uso principal
ESP-IDF	Framework principal de firmware
Toolchain Xtensa	Compilación para ESP32-S3
CMake + Ninja	Configuración y build
Python	Herramientas auxiliares e idf.py
Monitor serial	Diagnóstico de arranque y pruebas
Editor recomendado	VS Code con extensión ESP-IDF
Git	Versionado de firmware y documentación

TODO técnico: La versión exacta del stack gráfico, la configuración definitiva del USB device y el modo final de SD (SDSPI o SDMMC) deben validarse contra el PCB definitivo.

4. Project Overview

La plataforma está organizada por capas para desacoplar hardware, servicios y lógica. Esto permite que cada componente crezca sin contaminar el resto del proyecto.

Capa	Responsabilidad
BSP / HAL	Mapeo de pines, buses y particularidades del board.
Drivers	Abstracción de periféricos y funciones de lectura/escritura.
Common	Tipos base, snapshots, enums y estructuras compartidas.

Services	Conectividad y servicios de configuración del sistema.
Communications	Modbus, MQTT y bridges hacia runtimes externos.
PLC Core	Ciclo de adquisición, snapshot y distribución interna.
Main	Orquestación del arranque y del lazo principal.
HMI / UI	Presentación local, navegación y widgets de usuario.

El flujo de información esperado es el siguiente: las señales físicas son capturadas por drivers; el PLC core consolida el estado del sistema en un snapshot; dicho snapshot alimenta bridges, servicios y HMI; los comandos del operador o de la red regresan al sistema por servicios y se reflejan en salidas, consignas o configuraciones.

> **Observación:** Aunque la arquitectura ya contempla HMI, varios módulos permanecen en fase skeleton/mock. Esto es intencional: el proyecto prioriza una base compilable, estable y documentada antes de cerrar la implementación hardware.

5. Project Structure

```
plc_firmware/
├─ CMakeLists.txt
├─ sdkconfig
├─ main/
│  ├─ CMakeLists.txt
│  └─ main.c
├─ components/
│  ├─ bsp/
│  ├─ common/
│  ├─ drivers/
│  │  ├─ i2c/
│  │  ├─ touch/
│  │  ├─ lcd/
│  │  ├─ adc_ext/
│  │  ├─ din/
│  │  ├─ dout/
│  │  ├─ energy/
│  │  ├─ ao_4_20ma/
│  │  ├─ ao_0_10v/
│  │  ├─ sdcard/
│  │  └─ usb_device/
│  ├─ comms/
│  │  ├─ modbus/
│  │  ├─ mqtt/
│  │  ├─ openplc_bridge/
│  │  └─ diac_bridge/
│  ├─ plc_core/
│  └─ services/
│     ├─ wifi_manager/
│     ├─ web_config_service/
│     └─ ble_config_service/
└─ build/
```

Directorio	Propósito
bsp	Board support package; contiene board_io_map y constantes de board.
common	Tipos base como plc_state_t, driver_health_t y plc_io_snapshot_t.
drivers	Módulos de acceso a periféricos y abstracciones de I/O.
comms	Protocolos y bridges de intercambio de datos.
services	Servicios de conectividad, provisión y configuración.
plc_core	Consolidación del snapshot y distribución a bridges.
main	Entrada del firmware y lazo de operación principal.

6. Drivers

Los drivers están diseñados para exponer una API pública clara, separando inicialización, operación, estado y diagnóstico. En la versión actual varios drivers se encuentran en estado skeleton o mock; aun así, la firma pública ya permite integrar servicios, HMI y PLC core sin depender todavía del hardware definitivo.

drv_i2c

Campo	Contenido
Propósito	Bus I2C compartido para periféricos de control y expansión.
Carpeta	drivers/i2c
Madurez actual	Bajo
Archivos típicos	include/drv_i2c.h · src/drv_i2c.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_i2c_init
- drv_i2c_deinit
- drv_i2c_probe
- drv_i2c_write
- drv_i2c_read
- drv_i2c_write_read
- drv_i2c_get_health

Recomendación de uso: Base para touch, expansores, ADC externo o periféricos futuros.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_touch

Campo	Contenido
Propósito	Captura de coordenadas táctiles y estado de pulsación.
Carpeta	drivers/touch
Madurez actual	Medio
Archivos típicos	include/drv_touch.h · src/drv_touch.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_touch_init
- drv_touch_update
- drv_touch_get_point
- drv_touch_mock_set
- drv_touch_get_health

Recomendación de uso: Debe amarrarse a controlador táctil real y a calibración de pantalla.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_lcd

Campo	Contenido
Propósito	Control básico de pantalla y backlight.
Carpeta	drivers/lcd
Madurez actual	Medio
Archivos típicos	include/drv_lcd.h · src/drv_lcd.c

Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_lcd_init
- drv_lcd_clear
- drv_lcd_set_backlight
- drv_lcd_draw_pixel
- drv_lcd_present
- drv_lcd_get_health

Recomendación de uso: Capa base; la librería gráfica debe apoyarse encima de este driver.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_adc_ext

Campo	Contenido
Propósito	Lectura de entradas analógicas externas con ganancia y offset.
Carpeta	drivers/adc_ext
Madurez actual	Medio
Archivos típicos	include/drv_adc_ext.h · src/drv_adc_ext.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_adc_ext_init
- drv_adc_ext_update
- drv_adc_ext_get_raw
- drv_adc_ext_get_scaled
- drv_adc_ext_mock_set
- drv_adc_ext_get_health

Recomendación de uso: Apto para simulación inicial y escalado temprano.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_din

Campo	Contenido
Propósito	Entradas digitales NPN/PNP con bitmap de estado.
Carpeta	drivers/din
Madurez actual	Bajo
Archivos típicos	include/drv_din.h · src/drv_din.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_din_init
- drv_din_update
- drv_din_get
- drv_din_get_bitmap

Recomendación de uso: Driver ya utilizable como base de integración.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_dout

Campo	Contenido
Propósito	Salidas digitales con acceso por canal y bitmap.
Carpeta	drivers/dout
Madurez actual	Bajo
Archivos típicos	include/drv_dout.h · src/drv_dout.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_dout_init
- drv_dout_set
- drv_dout_get
- drv_dout_get_bitmap

- drv_dout_write_bitmap

Recomendación de uso: Preparado para NPN, PNP y relés.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_energy

Campo	Contenido
Propósito	Interfaz para lector de energía y variables eléctricas.
Carpeta	drivers/energy
Madurez actual	Medio
Archivos típicos	include/drv_energy.h · src/drv_energy.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_energy_init
- drv_energy_update
- drv_energy_get_data
- drv_energy_mock_set
- drv_energy_get_health

Recomendación de uso: Actualmente permite simulación/placeholder de datos.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_ao_4_20ma

Campo	Contenido
Propósito	Generación de consigna analógica en corriente.
Carpeta	drivers/ao_4_20ma
Madurez actual	Medio

Archivos típicos	include/drv_ao_4_20ma.h · src/drv_ao_4_20ma.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_ao_4_20ma_init
- drv_ao_4_20ma_set_current
- drv_ao_4_20ma_set_engineering
- drv_ao_4_20ma_get_current
- drv_ao_4_20ma_get_health

Recomendación de uso: La conversión final depende del hardware analógico definitivo.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_ao_0_10v

Campo	Contenido
Propósito	Generación de consigna analógica en voltaje.
Carpeta	drivers/ao_0_10v
Madurez actual	Medio
Archivos típicos	include/drv_ao_0_10v.h · src/drv_ao_0_10v.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_ao_0_10v_init
- drv_ao_0_10v_set_voltage
- drv_ao_0_10v_set_engineering
- drv_ao_0_10v_get_voltage
- drv_ao_0_10v_get_health

Recomendación de uso: Escalado ya definido por API.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_sdcard

Campo	Contenido
Propósito	Montaje y supervisión de almacenamiento local.
Carpeta	drivers/sdcard
Madurez actual	Medio
Archivos típicos	include/drv_sdcard.h · src/drv_sdcard.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_sdcard_init
- drv_sdcard_mount
- drv_sdcard_unmount
- drv_sdcard_get_status
- drv_sdcard_get_health

Recomendación de uso: Pendiente de validar modo final SDSPI/SDMMC.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

drv_usb_device

Campo	Contenido
Propósito	Soporte USB en modo dispositivo (CDC/MSC/composite).
Carpeta	drivers/usb_device
Madurez actual	Medio
Archivos típicos	include/drv_usb_device.h · src/drv_usb_device.c
Dependencias	common + BSP + hardware específico según el caso

Funciones públicas disponibles:

- drv_usb_device_init
- drv_usb_device_start
- drv_usb_device_stop

- drv_usb_device_cdc_write
- drv_usb_device_cdc_read
- drv_usb_device_get_status
- drv_usb_device_get_health

Recomendación de uso: Debe integrarse con stack device definitivo.

Manejo de errores: la API utiliza esp_err_t y una estructura de health/status para desacoplar la capa de aplicación del detalle del periférico. Donde falte hardware definitivo, la implementación actual debe tratarse como base funcional, no como driver de producción.

7. BSP and I/O Mapping

El BSP encapsula los detalles del board y evita que el resto del firmware use números de GPIO dispersos en el código. El archivo central es board_io_map.h, donde se asignan nombres lógicos a los recursos físicos.

Señal lógica	Asignación actual
I2C SDA	GPIO17
I2C SCL	GPIO18
RS485 RX	GPIO8
RS485 TX	GPIO9
DI NPN1..4	GPIO1, GPIO2, GPIO3, GPIO4
DI PNP1..2	GPIO5, GPIO6
DO NPN1..4	GPIO35, GPIO36, GPIO37, GPIO38
DO RLY1..2	GPIO41, GPIO42
DO PNP1..2	GPIO46, GPIO47

Pendiente de validar: El mapeo actual cubre I²C, RS485, DI y DO. Las señales de pantalla, USB, SD, analógicos y algunos buses auxiliares deben cerrarse definitivamente cuando se valide el PCB y la tarjeta mezzanine.

La recomendación de ingeniería es mantener dos capas separadas: board_io_map.h para la definición física y estructuras de configuración por driver para la adaptación funcional. Esto reduce el impacto de un cambio de PCB.

8. PLC Core

plc_core es la pieza que transforma drivers dispersos en una imagen coherente del sistema. Su responsabilidad no es manejar UI ni protocolos de red, sino consolidar el estado del proceso.

- Solicitar actualización de drivers relevantes.
- Construir un plc_io_snapshot_t consistente.
- Publicar el snapshot a openplc_bridge y diac_bridge.
- Servir como punto natural de conexión futura hacia Modbus, MQTT y HMI.

Elemento del snapshot	Origen
di_bitmap	drv_din_get_bitmap()
do_bitmap	drv_dout_get_bitmap()
ai[]	drv_adc_ext_get_scaled()
ao[]	drv_ao_0_10v_get_voltage() / AO equivalente
energy_voltage/current/power/frequency	drv_energy_get_data()

Estados globales: Los tipos plc_state_t definen BOOT, INIT, STOP, RUN, MANUAL, MAINTENANCE, WARNING y ERROR. La política completa de transición entre estados sigue siendo un TODO técnico a formalizar en el siguiente ciclo de desarrollo.

9. Communications

Modbus

Campo	Contenido
Propósito	Exponer variables y consignas del PLC a clientes RTU/TCP.
Módulos	modbus_service + modbus_register_map
Datos principales	Coils, discrete inputs, input registers y holding registers.
Estado actual	Base funcional / integración temprana

Diagnóstico básico: cada servicio debe poder exponer estado de running, contadores y último error. La instrumentación temprana es más importante en esta fase que la implementación final del protocolo.

MQTT

Campo	Contenido

Propósito	Publicar estado del PLC y recibir comandos o consignas.
Módulos	mqtt_service
Datos principales	URI de broker, client_id, topic_pub, topic_sub.
Estado actual	Base funcional / integración temprana

Diagnóstico básico: cada servicio debe poder exponer estado de running, contadores y último error. La instrumentación temprana es más importante en esta fase que la implementación final del protocolo.

OpenPLC bridge

Campo	Contenido
Propósito	Recibir snapshots desde el PLC core y acercarlos a un runtime externo.
Módulos	openplc_bridge
Datos principales	runtime_state, cycle_count, last_error.
Estado actual	Base funcional / integración temprana

Diagnóstico básico: cada servicio debe poder exponer estado de running, contadores y último error. La instrumentación temprana es más importante en esta fase que la implementación final del protocolo.

4Diac bridge

Campo	Contenido
Propósito	Canal equivalente para integración con una arquitectura IEC 61499.
Módulos	diac_bridge
Datos principales	runtime_state, event_count, last_error.
Estado actual	Base funcional / integración temprana

Diagnóstico básico: cada servicio debe poder exponer estado de running, contadores y último error. La instrumentación temprana es más importante en esta fase que la implementación final del protocolo.

Web config service

Campo	Contenido

Propósito	Exponer configuración por interfaz web.
Módulos	web_config_service
Datos principales	Puerto, autenticación, contador de requests.
Estado actual	Base funcional / integración temprana

Diagnóstico básico: cada servicio debe poder exponer estado de running, contadores y último error. La instrumentación temprana es más importante en esta fase que la implementación final del protocolo.

WiFi manager

Campo	Contenido
Propósito	Gestionar modo AP, STA o APSTA y estado de conectividad.
Módulos	wifi_manager
Datos principales	SSIDs, contraseña, canal AP, estado STA/AP.
Estado actual	Base funcional / integración temprana

Diagnóstico básico: cada servicio debe poder exponer estado de running, contadores y último error. La instrumentación temprana es más importante en esta fase que la implementación final del protocolo.

BLE config service

Campo	Contenido
Propósito	Provisión o configuración básica vía BLE.
Módulos	ble_config_service
Datos principales	device_name, enable_provisioning, sesiones.
Estado actual	Base funcional / integración temprana

Diagnóstico básico: cada servicio debe poder exponer estado de running, contadores y último error. La instrumentación temprana es más importante en esta fase que la implementación final del protocolo.

Modbus: La definición de registros es obligatoria y ya existe una base en modbus_register_map.h. Esta tabla deberá versionarse cuando el equipo defina el mapa definitivo de variables.

10. HMI Overview

La HMI está concebida como una interfaz compacta, horizontal, de 320 px de ancho, con lenguaje visual industrial moderno. El objetivo no es reproducir una tablet genérica, sino ofrecer una superficie de control legible, rápida y consistente con un panel de automatización.

- Fondo oscuro de alto contraste.
- Acentos violeta/azul con iluminación de borde.
- Menú horizontal fijo con iconos compactos.
- Encabezado superior con estado global, IP y hora.
- Widgets y tarjetas con jerarquía visual marcada.
- Textos breves y operables en espacio reducido.

Referencias visuales del proyecto:

Inspiración visual: Las referencias de pantalla incluidas en el proyecto muestran un dashboard oscuro con barras superiores iluminadas, pantallas de estados E/S, configuración de red, comunicación y alarmas. La implementación final debe respetar esa dirección estética.

11. HMI Screens

Inicio

- Objetivo: ofrecer una visión general del estado del PLC en menos de cinco segundos.
- Contenido mínimo: estado RUN/STOP/ERROR, IP, hora/fecha, alarmas resumidas, entradas/salidas activas, energía o variable principal y accesos rápidos.
- Fuentes de datos: plc_core snapshot, servicios de conectividad y gestor de alarmas.
- Nivel de usuario: operador.
- Widgets sugeridos: tarjetas de estado, indicador de alarmas, botones rápidos y resumen energético.

Estados

- Objetivo: monitoreo detallado de DI, DO, AI, AO, energía y variables de proceso.
- Contenido mínimo: nombre de señal, valor, unidad, estado de color y subtipo de vista (digital, analógico, energía).
- Fuentes de datos: drivers y snapshot consolidado.
- Nivel de usuario: operador / técnico.
- Widgets sugeridos: lista dual de entradas y salidas, vistas por pestañas y colores de estado.

Control

- Objetivo: cambiar modos de operación y aplicar acciones manuales permitidas.
- Contenido mínimo: selección manual/automático, consignas, confirmación de acciones críticas, bloqueo por rol.
- Fuentes de datos: PLC core y servicios de permisos.
- Nivel de usuario: técnico / administrador.

Configuración

- Objetivo: gestionar parámetros de red, nombres de señales, escalados y ajustes de sistema.
- Contenido mínimo: IP, máscara, gateway, DNS, DHCP, puerto web, fecha/hora, brillo/contraste y permisos.
- Fuentes de datos: servicios de configuración y módulos de red.
- Nivel de usuario: técnico / administrador.

Comunicación

- Objetivo: mostrar salud de red, protocolos y servicios remotos.
- Contenido mínimo: estado Ethernet/Wi-Fi, estado Modbus, estado MQTT, web server, TX/RX y último error.
- Fuentes de datos: wifi_manager, web_config_service, modbus_service, mqtt_service, bridges.
- Nivel de usuario: técnico.

Alarmas

- Objetivo: concentrar alarmas activas, prioridad e historial.
- Contenido mínimo: lista de eventos, severidad, hora de activación, reconocimiento, reset e historial.
- Fuentes de datos: gestor de alarmas (pendiente de formalizar).
- Nivel de usuario: operador / técnico.

Mantenimiento

- Objetivo: pruebas, diagnóstico y recuperación del sistema.
- Contenido mínimo: test de E/S, LCD, touch, ADC, reinicio de servicios, backup, restauración y reset de fábrica.
- Fuentes de datos: drivers, servicios y rutinas de diagnóstico.

- Nivel de usuario: técnico / administrador.

Sistema

- Objetivo: inventario técnico del equipo y versión del stack.
- Contenido mínimo: nombre del proyecto, versión de firmware, versión de hardware, memoria, uptime, temperatura, fecha de build y versiones de drivers.
- Fuentes de datos: build info, common, servicios de diagnóstico.
- Nivel de usuario: técnico / administrador.

12. Build and Flash

El proyecto usa ESP-IDF con CMake y Ninja. La configuración de componentes se realiza desde el CMakeLists raíz mediante EXTRA_COMPONENT_DIRS para cada subcomponente del árbol.

```
idf.py set-target esp32s3
idf.py reconfigure
idf.py build
idf.py flash
idf.py -p COMx flash monitor
```

Etapa	Propósito
set-target	Fijar el target a ESP32-S3
reconfigure	Regenerar CMake después de cambios estructurales
build	Compilar aplicación, bootloader y tablas
flash	Programar firmware
monitor	Supervisar logs de arranque y diagnóstico

Errores comunes: Si aparece un componente desconocido, revisar EXTRA_COMPONENT_DIRS, REQUIRES y la presencia de CMakeLists.txt dentro del componente. Si fallan includes, validar INCLUDE_DIRS y el nombre exacto del componente.

Ruta del proyecto: Aunque la evidencia de build actual demuestra que el proyecto ya compila en el entorno de trabajo disponible, se recomienda evitar rutas con espacios durante integración, scripts y automatización futura.

13. Configuration and Operation

Flujo de primera configuración sugerido:

- Arranque inicial del firmware y verificación del estado base.
- Configuración de red local (AP o STA/AP según política final).
- Acceso al web server de configuración.

- Opcionalmente, provisión inicial por BLE.
- Configuración de parámetros Modbus y MQTT.
- Ajuste de nombres de señales, escalados y permisos.
- Reinicio controlado y entrada en modo RUN o mantenimiento.

El flujo de operación normal esperado es: arranque → init de drivers → arranque de servicios → init de plc_core → actualización periódica del snapshot → publicación a bridges y exposición a la HMI.

Modo mantenimiento: Debe usarse para pruebas de E/S, calibración y diagnóstico. La entrada a este modo debe estar protegida por permisos.

14. Troubleshooting

Problema	Causa probable	Acción sugerida
No compila	Componente no encontrado, include roto, CMake incompleto	Revisar CMakeLists, REQUIRES, EXTRA_COMPONENT_DIRS y limpiar build.
No flashea	Puerto incorrecto, driver USB, alimentación inestable	Verificar COM, cable y modo de programación.
Pantalla no inicializa	Pinout pendiente, driver LCD incompleto, backlight no habilitado	Validar board_io_map, secuencia init y alimentación de display.
Touch no responde	Bus/timing pendiente, orientación no calibrada	Verificar controlador touch, ejes y driver.
MQTT no conecta	URI o credenciales incorrectas	Revisar broker_uri, red y puertos.
Modbus no responde	Mapa de registros, ID o modo incorrecto	Verificar slave_id, dirección y tabla de registros.
IP incorrecta	Configuración no persistida o parámetro inválido	Revisar web config, BLE config o política de red.
Alarmas permanentes	No existe gestor final o umbrales no ajustados	Ajustar política de alarmas y fuentes de datos.
Valores analógicos fuera de rango	Ganancia/offset o scaling sin calibrar	Revisar cfg del ADC y AO.
SD o USB no monta	Modo no validado, stack no conectado al hardware real	Validar implementación final del driver y esquema físico.
Web config no abre	Wi-Fi no iniciado o servicio detenido	Comprobar wifi_manager y web_config_service.

15. Development Notes

- Mantener nombres de componentes coherentes con el directorio que los contiene.
- Evitar lógica de control en main; usar main solo como orquestador de arranque.
- Toda nueva señal física debe reflejarse primero en board_io_map.h.
- Toda nueva API pública debe devolver esp_err_t o exponer estado compatible con driver_health_t.
- HMI y comunicaciones deben consumir snapshots, no leer hardware directo.
- Los módulos mock/skeleton deben marcarse con TODO técnico cuando haya riesgo de confusión.

Cómo agregar un nuevo driver: crear componente propio, definir header con estructuras de configuración, exponer init/update/read/write, agregar health/status, declarar CMakeLists y añadir el componente a EXTRA_COMPONENT_DIRS si corresponde.

Cómo agregar una nueva pantalla: definir la pantalla en la capa HMI, especificar fuentes de datos, permisos, eventos de navegación y widgets; evitar mezclar la lógica de presentación con el acceso directo al driver.

Cómo agregar un nuevo protocolo: crear componente en comms o services, definir configuración, estado, init/start/stop y vincularlo a un snapshot o a un mapa de variables compartidas.

16. Roadmap / Future Work

- Completar implementación real de LCD + Touch + librería gráfica.
- Formalizar gestor de alarmas y sistema de permisos por rol.
- Conectar modbus_service con snapshot y comandos del PLC core.
- Conectar mqtt_service con publicación periódica y recepción de comandos.
- Evolucionar web_config_service a servidor HTTP operativo con persistencia.
- Implementar BLE provisioning real.
- Cerrar la política de USB device (CDC, MSC o compuesto) y SD real.
- Agregar logging persistente, exportación a SD y respaldo de configuración.
- Incorporar pruebas unitarias y smoke tests por componente.

17. Glossary

Término	Definición
BSP	Board Support Package; encapsula detalles del hardware del board.
HAL	Hardware Abstraction Layer; nivel de abstracción sobre periféricos.
HMI	Human Machine Interface; interfaz local de operación.
PLC	Programmable Logic Controller; sistema de control programable.
DI / DO	Digital Input / Digital Output.
AI / AO	Analog Input / Analog Output.
Snapshot	Estructura consolidada del estado del sistema en un instante.

Modbus	Protocolo industrial para intercambio de variables y control.
MQTT	Protocolo publish/subscribe orientado a telemetría.
OpenPLC	Runtime/plataforma IEC 61131-3 usada como referencia o integración.
4Diac	Entorno orientado a IEC 61499 para automatización distribuida.
Provisioning	Proceso de configuración inicial de red o credenciales.

18. Related Documents

- Firmware source tree del proyecto plc_firmware.
- CMakeLists.txt raíz y CMakeLists por componente.
- board_io_map.h y tipos comunes del proyecto.
- Planificación visual de la HMI (pantallas de referencia).
- Documentación de build y logs de compilación del entorno ESP-IDF.
- Futuro documento de mapa Modbus definitivo.
- Futuro documento de políticas de alarmas, usuarios y mantenimiento.

Estado del documento: Este manual es una base técnica sólida y navegable para la etapa actual del proyecto. A medida que el hardware se confirme, deberá actualizarse la sección de BSP, HMI real, política de red, servicios finales y pruebas de campo.